

PROJECT REPORT

AutoJudge: Programming Problem Difficulty Prediction System

1. Introduction

Estimating the difficulty of programming problems is an important yet subjective task in competitive programming platforms and educational settings. Difficulty levels influence problem selection, contest design, and learning progression. Traditionally, difficulty assignment is performed manually by experts, which can be inconsistent and time-consuming.

This project, **AutoJudge**, aims to automate the estimation of programming problem difficulty using machine learning techniques. Given a problem's textual description, the system predicts:

- A **difficulty class** (Easy / Medium / Hard)
- A **numerical difficulty score** on a scale of **1–10**

The project integrates natural language processing, classical machine learning models, and a web-based interface for real-time prediction.

2. Problem Statement

The objective of this project is to design a system that can analyze the textual content of programming problems and automatically estimate their difficulty.

Formally, given:

- Problem title
- Problem description
- Input description
- Output description

the system should:

1. Classify the problem into **Easy**, **Medium**, or **Hard**
2. Predict a **difficulty score between 1 and 10**

The solution must be accurate, interpretable, and runnable locally through a web interface.

3. Dataset Description

A downloaded dataset (`dataset.csv`) was used for this project.

Dataset Attributes

The dataset contains the following fields:

- **Problem Title**
- **Problem Description**
- **Input Description**
- **Output Description**
- **Difficulty Class** (Easy / Medium / Hard)
- **Difficulty Score** (1–10)

Each row represents a single programming problem with both categorical and numerical difficulty labels.

4. Data Preprocessing

Before model training, several preprocessing steps were applied:

1. **Handling Missing Values**
All missing textual fields were replaced with empty strings to avoid errors during text processing.
 2. **Text Consolidation**
The title, problem description, input description, and output description were concatenated into a single text field for feature extraction.
 3. **Label Preparation**
 - Difficulty class was used for classification.
 - Difficulty score (1–10) was used for regression.
-

5. Feature Engineering

Two types of features were used:

5.1 TF-IDF Text Features

Term Frequency–Inverse Document Frequency (TF-IDF) was used to convert textual data into numerical vectors. This captures the importance of words relative to the entire dataset.

5.2 Handcrafted Numerical Features

Additional features were engineered to capture structural complexity:

- Length of the problem text
- Count of mathematical symbols
- Presence of algorithm-related keywords such as:
 - DP
 - Graph
 - Tree
 - BFS
 - DFS
 - Recursion
 - Greedy

These numerical features were scaled before being combined with TF-IDF features.

6. Model Architecture

6.1 Classification Model

- **Model Used:** Logistic Regression
- **Task:** Predict difficulty class (Easy / Medium / Hard)

Logistic Regression was chosen due to its simplicity, interpretability, and effectiveness for text-based classification tasks.

6.2 Regression Model

- **Model Used:** Gradient Boosting Regressor
- **Task:** Predict difficulty score (1–10)

Gradient Boosting was selected as it handles non-linear relationships well and performs robustly on mixed feature sets (text + numeric).

7. Experimental Setup

- Programming Language: Python
 - Libraries Used:
 - scikit-learn
 - pandas
 - numpy
 - scipy
 - joblib
 - streamlit
 - Evaluation performed using a held-out validation split during training.
 - Models were saved as `.pkl` files and loaded during inference.
-

8. Evaluation Metrics and Results

8.1 Classification Evaluation

- **Metric:** Accuracy
- A confusion matrix was generated to analyze class-wise performance.

Accuracy: 0.4823815309842041

Confusion Matrix:

```
[[ 79  19  38]
 [ 85 215 125]
 [ 58 101 103]]
```

Classification Report:

	precision	recall	f1-score	support
easy	0.36	0.58	0.44	136
hard	0.64	0.51	0.57	425
medium	0.39	0.39	0.39	262
accuracy			0.48	823
macro avg	0.46	0.49	0.47	823
weighted avg	0.51	0.48	0.49	823

8.2 Regression Evaluation

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)

These metrics measure how close the predicted difficulty score is to the actual score.

MAE: 1.6743733530402036

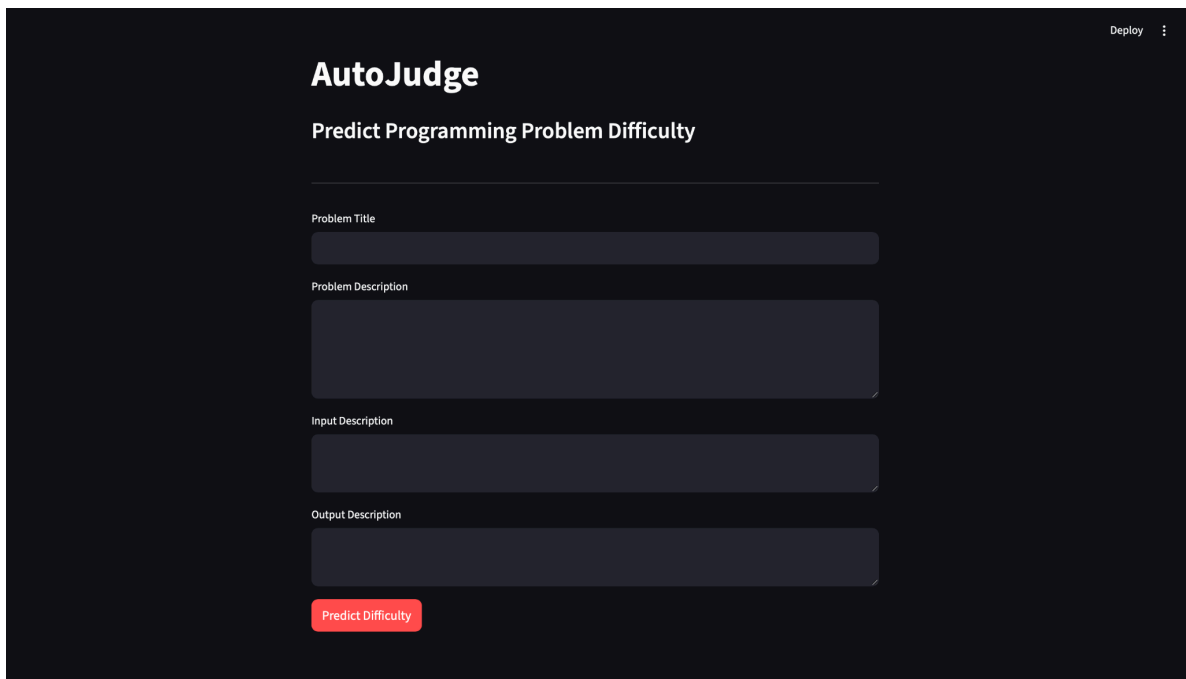
RMSE: 2.0241765994505365

9. Web Interface

A web interface was developed using **Streamlit** to allow interactive use of the system.

Features:

- User inputs problem title, description, input, and output
- On clicking “Predict Difficulty”:
 - Difficulty class is displayed
 - Numerical difficulty score is shown



The screenshot displays the 'AutoJudge' web interface on a dark background. At the top right, there is a 'Deploy' button with a dropdown arrow. The main heading is 'AutoJudge' in a large, bold font, followed by the subtitle 'Predict Programming Problem Difficulty'. Below this, there are four input fields with labels: 'Problem Title', 'Problem Description', 'Input Description', and 'Output Description'. Each field is represented by a dark gray rounded rectangle. At the bottom of the form is a red button with the text 'Predict Difficulty'.

The web app loads pre-trained models and performs real-time inference locally.

10. Sample Predictions

Several sample programming problems were tested through the web interface to validate system behavior.

This screenshot shows the web interface for the 'Shortest Path in a Weighted Graph' problem. The interface is dark-themed and includes a 'Deploy' button in the top right corner. The problem title is 'Shortest Path in a Weighted Graph'. The 'Problem Description' states: 'You are given a connected weighted graph with N nodes and M edges. Each edge has a positive weight. Your task is to compute the shortest distance from a given source node S to all other nodes in the graph. The graph does not contain negative weight edges. Efficient handling of large graphs is required.' The 'Input Description' states: 'The next M lines each contain three integers u, v, and w, representing an edge between nodes u and v with weight w. The last line contains an integer S, the source node.' The 'Output Description' states: 'Print N integers where the i-th integer represents the shortest distance from node S to node i.' Below the descriptions is a red 'Predict Difficulty' button. The 'Prediction Results' section shows a 'Predicted Class' of 'MEDIUM' and a 'Predicted Score' of '5.38'.

Shortest Path in a Weighted Graph

Problem Description

You are given a connected weighted graph with N nodes and M edges. Each edge has a positive weight. Your task is to compute the shortest distance from a given source node S to all other nodes in the graph. The graph does not contain negative weight edges. Efficient handling of large graphs is required.

Input Description

The next M lines each contain three integers u, v, and w, representing an edge between nodes u and v with weight w. The last line contains an integer S, the source node.

Output Description

Print N integers where the i-th integer represents the shortest distance from node S to node i.

Predict Difficulty

Prediction Results

Predicted Class	Predicted Score
MEDIUM	5.38

This screenshot shows the web interface for the 'Sum of Two Numbers' problem. The interface is dark-themed and includes a 'Deploy' button in the top right corner. The problem title is 'Sum of Two Numbers'. The 'Problem Description' states: 'You are given two integers A and B. Your task is to compute their sum.' The 'Input Description' states: 'The input consists of a single line containing two integers A and B.' The 'Output Description' states: 'Print a single integer representing the sum of the given two numbers.' Below the descriptions is a red 'Predict Difficulty' button. The 'Prediction Results' section shows a 'Predicted Class' of 'EASY' and a 'Predicted Score' of '2.64'.

Sum of Two Numbers

Problem Description

You are given two integers A and B. Your task is to compute their sum.

Input Description

The input consists of a single line containing two integers A and B.

Output Description

Print a single integer representing the sum of the given two numbers.

Predict Difficulty

Prediction Results

Predicted Class	Predicted Score
EASY	2.64

These examples demonstrate that the model provides reasonable relative difficulty estimates based on textual complexity.

11. Conclusion

This project successfully demonstrates that machine learning techniques can be used to estimate programming problem difficulty from textual descriptions. By combining TF-IDF features with handcrafted indicators and classical ML models, the system provides both categorical and numerical difficulty predictions.

While difficulty estimation remains subjective, AutoJudge offers a consistent and automated approach that can assist educators, problem setters, and learners.

12. Future Work

Possible improvements include:

- Expanding the dataset for better generalization
- Incorporating algorithm tags as additional inputs
- Improving regression calibration